Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка приложения по варианту ДЭ
и реализация дипломного проекта»

студента IV курса 285 группы

Фамилия Васильева

Имя, отчество Анастасия Александровна

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение 5
1 Демонстрационный экзамен 6
1.1 Анализ предметной области и технического задания 6
1.2 Выбор стека технологий и инструментов разработки 6
1.3 Работа с базой данных 8
1.4 Практическая реализация решения 9
1.5 Интерфейс программы 11
1.6 Тестирование программы 15
1.7 Руководство системному программисту 16
1.7.1 Общие сведения о программе 16
1.7.2 Функции программы 16
1.7.3 Минимальный состав технических средств 17
1.7.4 Минимальный состав программных средств 17
1.7.5 Требования к персоналу (системному программисту) 18
1.8 Структура программы 18
1.8.1 Сведения о составных частях программы 18
1.8.2 Сведения о связях между составными частями программы 18
1.8.3 Сведения о связях с другими программами 20
1.9 Настройка программы 20
1.9.1 Настройка на состав технических средств 20
1.9.2 Настройка на состав программных средств 20
1.10 Проверка программы 21
1.10.1 Описание способов проверки 21
1.11 Сообщения системному программисту 21
2 Дипломный проект 23
2.1 Введение дипломного проекта 23
2.2 Анализ предметной области 23
2.3 Требование к своему программному продукту 25
2.4 Обоснование выбора инструментальных средств разработки 25
2.5 Разработка архитектуры блок-схем программы 27
2.6 Разработка макетов основных форм приложения 29
2.7 Проектирование и разработка базы данных 31
2.8 Разработка пользовательского интерфейса 33
2.9 Доработка приложения 38
2.9.1 Разработка блок-схем функций 38

2.9.2 Разработка функционала 41
Заключение 45
Библиография 46
Приложение А 47

Введение

В рамках учебной практики будет выполнена разработка программного модуля для учёта заявок на ремонт климатического оборудования ООО «Конди». Практика проводилась в форме демонстрационного экзамена по варианту № 5.

Актуальность разработки обусловлена тем, что ведение учёта заявок на ремонт вручную или в электронных таблицах приводит к потере данных, затруднённому поиску информации и невозможности одновременной работы нескольких сотрудников. Автоматизация данного процесса позволяет повысить эффективность работы сервисного центра и качество обслуживания клиентов.

Целью практики является разработка полнофункционального веб-приложения для автоматизации учёта заявок на ремонт климатического оборудования.

Для достижения поставленной цели необходимо было решить следующие задачи:

проанализировать предметную область и определить основные бизнес-процессы сервисного центра по ремонту климатического оборудования;

разработать структуру базы данных в третьей нормальной форме, обеспечивающую хранение информации о пользователях, заявках и комментариях;

реализовать разграничение прав доступа для пяти категорий пользователей: Администратора, Менеджера, Оператора, Специалиста и Заказчика;

разработать удобный веб-интерфейс для работы с заявками, включая создание, редактирование, фильтрацию и поиск;

провести функциональное тестирование разработанного программного продукта и оформить соответствующую документацию.

1 Демонстрационный экзамен

1.1 Анализ предметной области и технического задания

ООО «Конди» — организация, специализирующаяся на продаже и обслуживании климатического оборудования. Для выполнения ремонтных работ привлекается сервисный центр «IT-Сфера», который осуществляет приём, обработку и исполнение заявок от заказчиков.

В рамках деятельности организации задействованы следующие категории сотрудников: операторы, принимающие заявки от клиентов; специалисты по ремонту климатического оборудования; менеджеры по качеству, контролирующие исполнение заявок; администраторы системы, управляющие учётными записями пользователей. Помимо этого, заказчики имеют возможность самостоятельно подавать заявки через систему.

До внедрения автоматизированной системы учёт заявок велся в бумажном виде или в электронных таблицах, что приводило к следующим проблемам: потеря информации о заявках, отсутствие оперативного контроля статуса ремонта, невозможность одновременной работы нескольких сотрудников с данными, трудоёмкость формирования отчётности..

1.2 Выбор стека технологий и инструментов разработки

Для разработки серверной части приложения был выбран язык программирования Python версии 3.13. Python — высокоуровневый язык программирования, отличающийся лаконичным синтаксисом, широкой экосистемой библиотек и активным сообществом разработчиков.

Обоснование выбора Python:

- простой и читаемый синтаксис ускоряет разработку и снижает вероятность ошибок;
- большое количество готовых библиотек для веб-разработки, работы с базами данных и криптографии;
- кроссплатформенность — приложение работает на Windows и Linux без изменений кода;
- широкое применение в веб-разработке, что обеспечивает доступность документации и готовых решений.

В качестве альтернативы рассматривался язык C#, однако он уступает Python по скорости разработки и требует более сложной инфраструктуры для веб-приложений.

В качестве системы управления базами данных выбрана SQLite 3. SQLite — встраиваемая реляционная СУБД, не требующая отдельного сервера и хранящая все данные в одном файле на диске.

Обоснование выбора SQLite:

- не требует установки и настройки отдельного сервера баз данных;
- поддержка полноценного SQL, включая JOIN, агрегатные функции, транзакции ACID;
- встроена в стандартную библиотеку Python — не требует дополнительной установки;
- простота резервного копирования — достаточно скопировать файл kondee.db;
- высокая скорость операций чтения, типичных для задач формирования отчётов.

Альтернативой является PostgreSQL. Однако при масштабе данного проекта (небольшой сервисный центр) PostgreSQL избыточна и требует дополнительных ресурсов для администрирования.

Разработка выполнялась в среде PyCharm. PyCharm — это специализированная IDE для языка Python, разработанная компанией JetBrains.

Преимущества PyCharm для данного проекта:

- интеллектуальное автодополнение кода с учётом типов данных;
- встроенный отладчик для пошагового выполнения кода;
- встроенная поддержка Flask: автоматическое распознавание маршрутов;
- удобный инструмент для работы с базой данных SQLite;
- интеграция с системой контроля версий Git.

Для реализации серверной логики веб-приложения выбран фреймворк Flask версии 3.x. Flask — это лёгкий микрофреймворк для Python, предоставляющий минимальный набор инструментов для создания веб-приложений.

Достоинства Flask:

- минималистичная архитектура без избыточного функционала;
- гибкость: разработчик самостоятельно выбирает необходимые компоненты;
- простая и понятная маршрутизация HTTP-запросов с помощью декораторов;
- встроенный шаблонизатор Jinja2 для генерации HTML-страниц;
- большое количество расширений для аутентификации, работы с БД и других задач.

В качестве альтернативы рассматривался фреймворк Django. Несмотря на более богатый встроенный функционал, Django избыточен для данного проекта: его панель администратора, ORM и встроенная система аутентификации усложнили бы разработку без существенных преимуществ. Flask позволяет реализовать именно тот функционал, который требуется по заданию.

Интерфейс веб-приложения реализуется с помощью стандартных технологий:

- HTML5 (для структурирования веб-страниц);

– CSS3 (для оформления и стилизации элементов);
– JavaScript (для интерактивного взаимодействия пользователя с системой);

Такое сочетание технологий обеспечивает современный внешний вид, простоту восприятия и корректное отображение интерфейса.

1.3 Работа с базой данных

База данных разработана в третьей нормальной форме (3НФ): все атрибуты зависят только от первичного ключа, транзитивных зависимостей нет. База данных содержит три таблицы.

Таблица users — хранит данные всех пользователей системы:

- user_id (PK) — уникальный идентификатор пользователя, AUTOINCREMENT;
- fio — полное имя пользователя, NOT NULL;
- phone — контактный телефон, NOT NULL;
- login — уникальный логин для входа в систему, UNIQUE NOT NULL;
- password — хеш пароля SHA-256, NOT NULL;
- role — роль пользователя с ограничением CHECK по допустимым значениям.

Таблица requests — хранит заявки на ремонт:

- request_id (PK) — уникальный номер заявки, AUTOINCREMENT;
- start_date — дата создания заявки, NOT NULL;
- climate_tech_type — тип климатического оборудования, NOT NULL;
- climate_tech_model — модель оборудования, NOT NULL;
- problem_description — описание неисправности, NOT NULL;
- request_status — статус заявки с ограничением CHECK, DEFAULT 'Новая заявка';
- completion_date — дата завершения ремонта, заполняется автоматически;
- repair_parts — перечень использованных комплектующих;
- client_id (FK) — внешний ключ на users(user_id), NOT NULL;
- master_id (FK) — внешний ключ на users(user_id), nullable (специалист не назначен).

Таблица comments — хранит комментарии специалистов к заявкам:

- comment_id (PK) — уникальный идентификатор комментария;
- message — текст комментария, NOT NULL;
- created_at — дата и время создания, DEFAULT datetime('now');
- master_id (FK) — ссылка на пользователя-автора комментария, NOT NULL;
- request_id (FK) — ссылка на заявку, к которой относится комментарий, NOT NULL.

Связи между таблицами: таблица requests связана с таблицей users двумя внешними ключами — client_id (заказчик, обязателен) и master_id (специалист, необязателен). Таблица comments связана с таблицами requests и users через внешние ключи request_id и master_id соответственно. Все связи имеют тип «один ко многим» (1:N).

Для ускорения поиска созданы индексы по полям: request_status, client_id, master_id, start_date (таблица requests) и request_id (таблица comments).

На рисунке 1.1 представлена ER-диаграмма сущностей и таблиц базы данных.

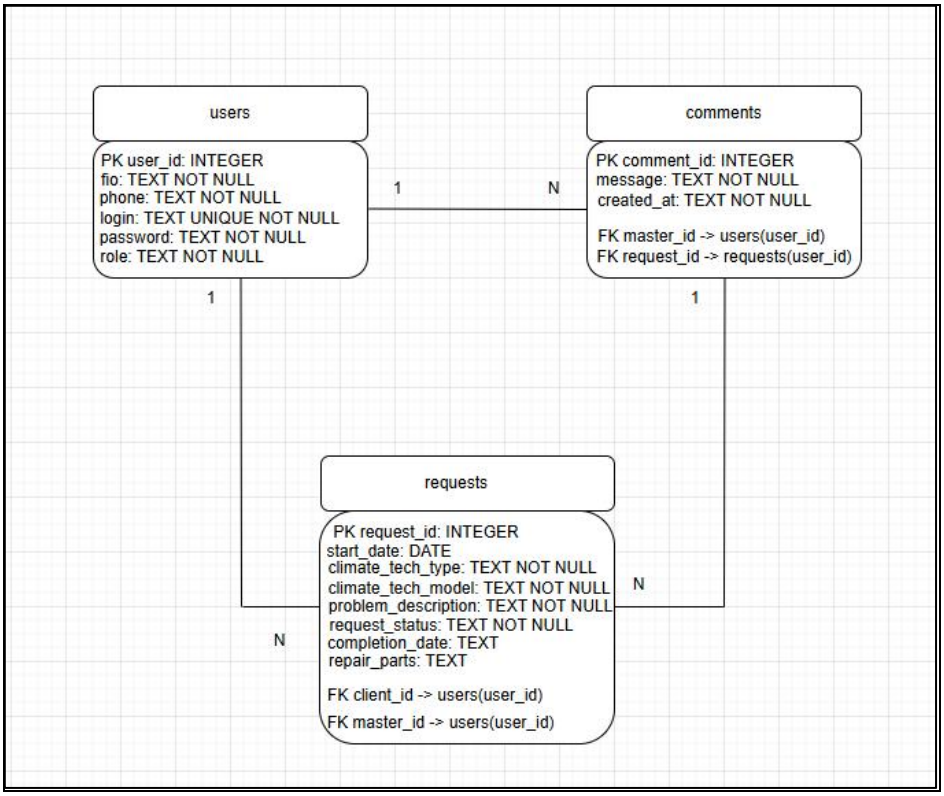

Рисунок 1.1 – ERD-диаграмма

Данные таблиц базы данных представлены в приложении А.

1.4 Практическая реализация решения

В соответствии с требованиями технического задания, перед непосредственным написанием программного кода был проведен этап проектирования логики работы приложения. Для наглядной визуализации процессов обработки данных были разработаны графические алгоритмы (блок-схема), выполненные в строгом соответствии с правилами оформления по ГОСТ 19.701.

Блок-схема описывает базовый алгоритм работы системы учета заявок на ремонт климатического оборудования. Она наглядно демонстрирует общий жизненный цикл обращения: получение входных данных от клиента при регистрации, проверку условий, изменение статусов заявки и формирование выходных данных при завершении ремонта (Рисунок 1.2)

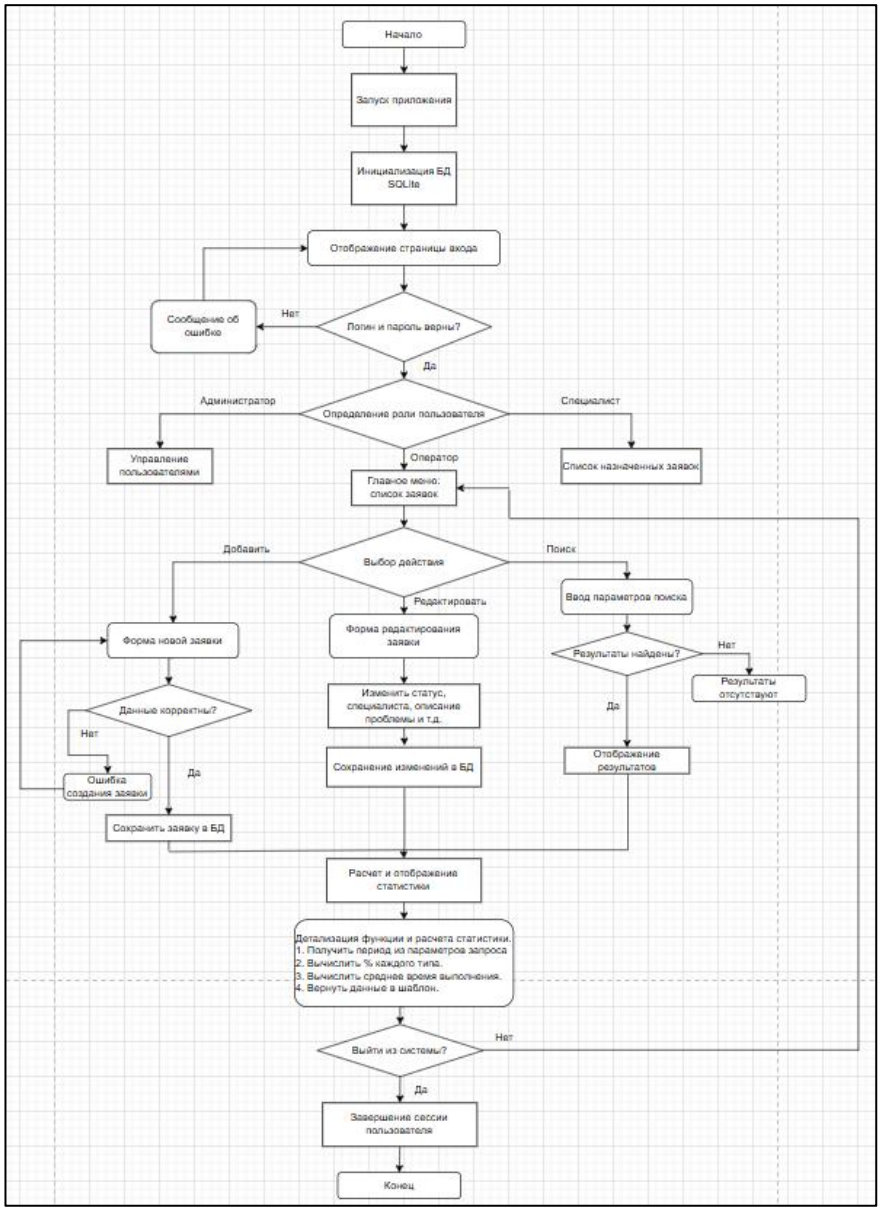

Рисунок 1.2 – Основная блок-схема

1.5 Интерфейс программы

Интерфейс приложения разработан с использованием HTML5, CSS3 и JavaScript. Цветовая схема основана на тёмно-синем цвете. Все страницы наследуют базовый шаблон base.html, содержащий шапку с навигацией.

Страница авторизации — точка входа в систему. Содержит поля ввода логина и пароля, кнопку входа и информационное сообщение при неверных данных. Страница доступна без авторизации, переход на другие страницы без входа невозможен (Рисунок 1.3).

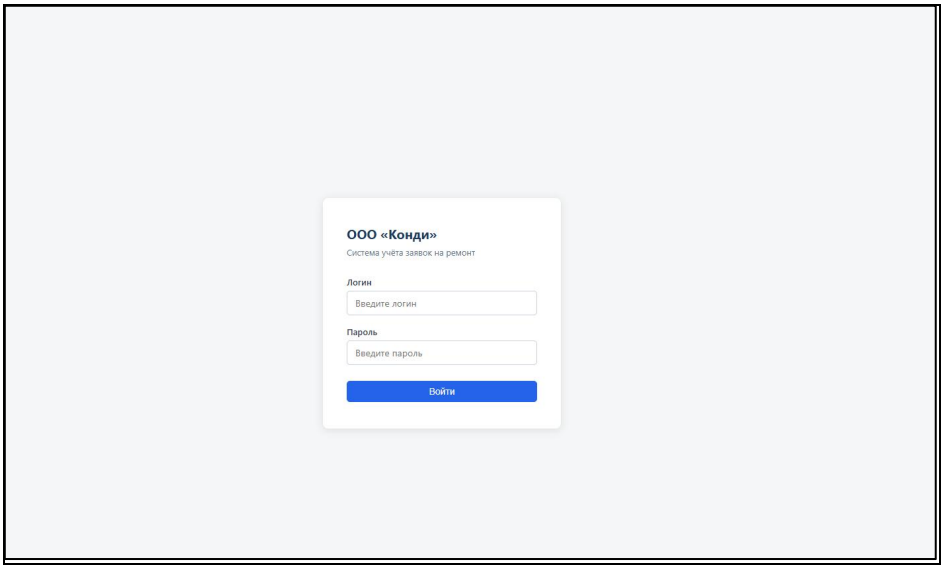

Рисунок 1.3 – Авторизация

Главная страница (список заявок) отображает все доступные пользователю заявки в виде таблицы. (Рисунок 1.4). Для Заказчика отображаются только его заявки, для остальных ролей — все. Предусмотрена фильтрация по статусу и типу оборудования, а также поиск по тексту. Статусы заявок отображаются в виде цветных бейджей:

- «Новая заявка» — серый;
- «В процессе ремонта» — синий;
- «Ожидание комплектующих» — жёлтый;
- «Готова к выдаче» — зелёный;

«Завершена» — тёмно-серый.

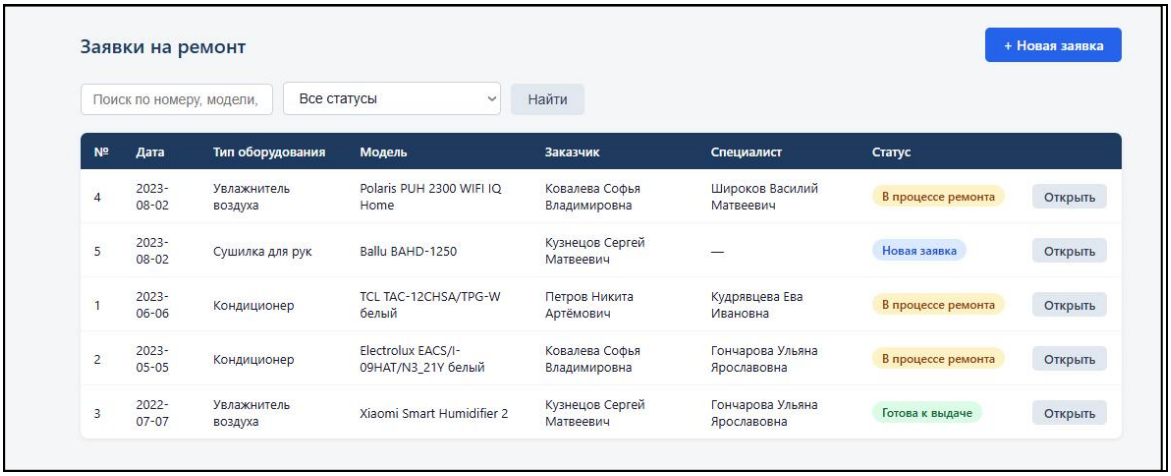

Рисунок 1.4 – Главная страница

Форма создания заявки позволяет выбрать тип оборудования из выпадающего списка, ввести модель и описание неисправности. Дата создания устанавливается автоматически. Форма доступна Оператору, Администратору и Заказчику (Рисунок 1.5).

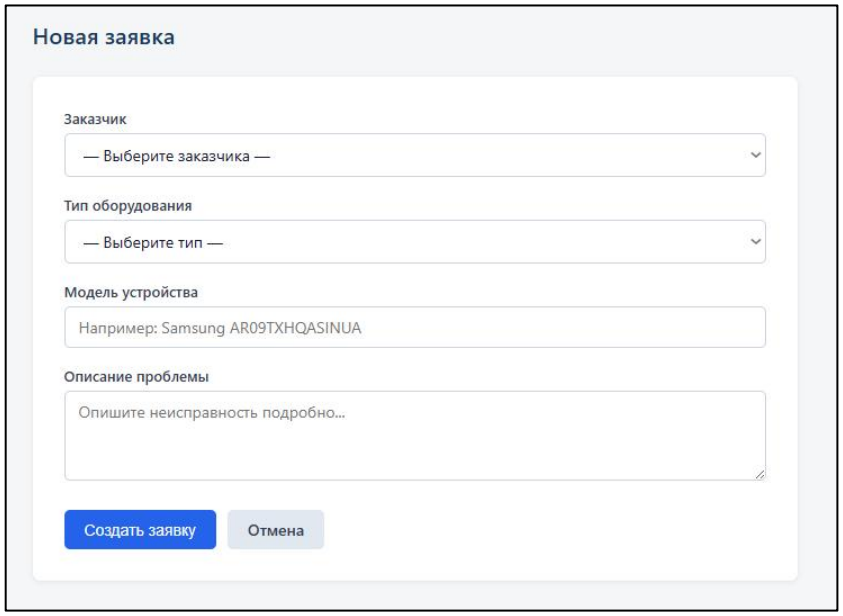

Рисунок 1.5 – Добавление новой заявки

Карточка заявки отображает полную информацию о заявке: реквизиты, статус, данные заказчика и специалиста, комплектующие, дату завершения. В нижней части карточки отображаются все комментарии с указанием автора и времени, а также форма для добавления нового комментария (Рисунок 1.6).

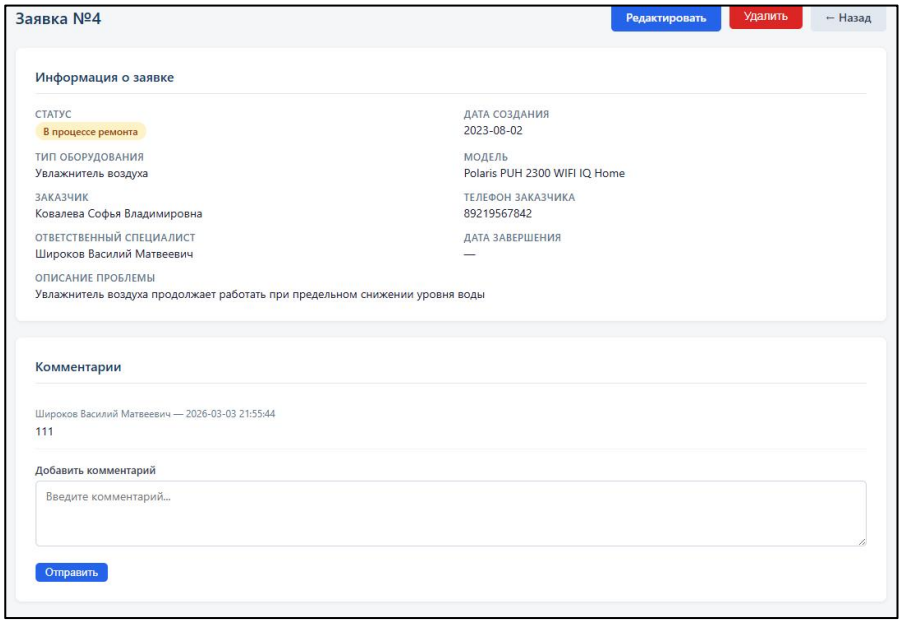

Рисунок 1.6 – Карточка заявки

Форма редактирования заявки позволяет изменить статус, назначить специалиста, добавить комплектующие. При установке статуса «Завершена» дата завершения устанавливается автоматически (Рисунок 1.7).

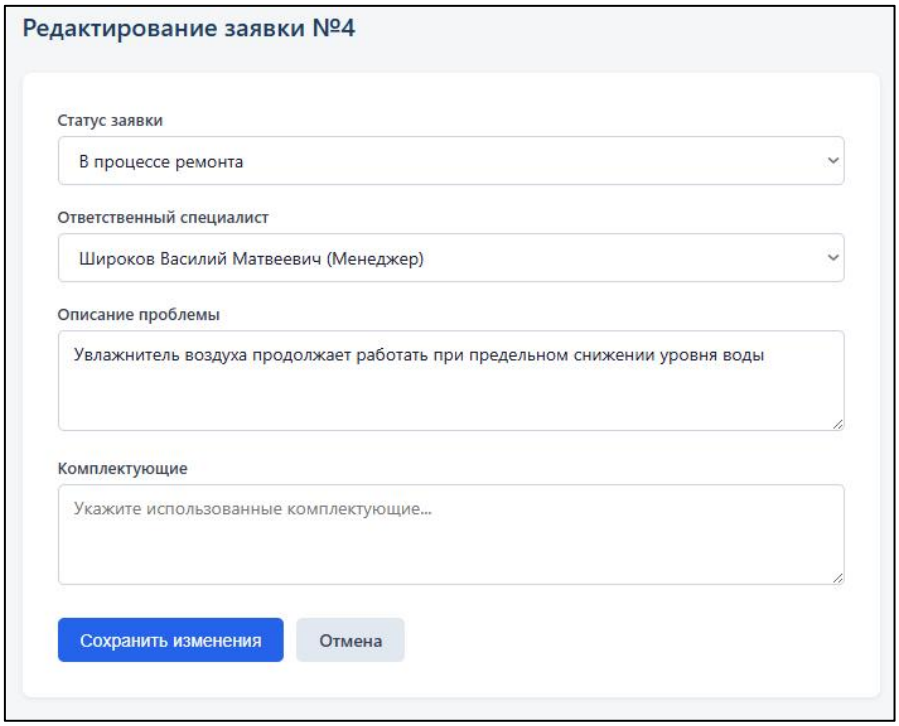

Рисунок 1.7 – Редактирование заявки

Страница статистики отображает сводную информацию: общее количество заявок, распределение по статусам и типам оборудования. Доступна Администратору, Менеджеру и Оператору (Рисунок 1.8).

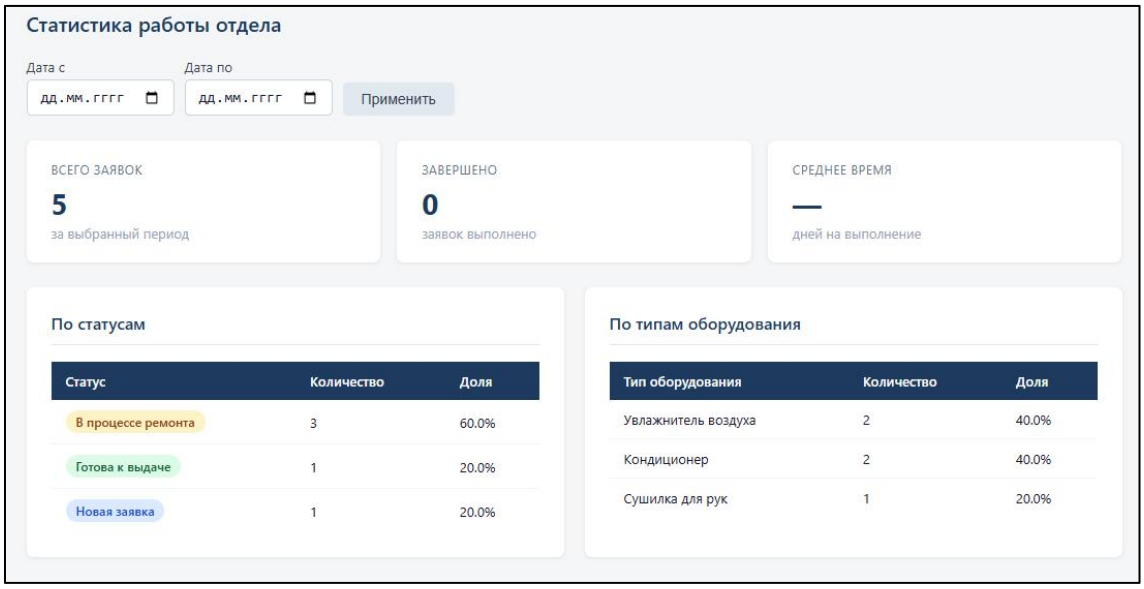

Рисунок 1.8 – Страница статистики

Страница управления пользователями отображает список всех зарегистрированных пользователей с их ролями. Позволяет создавать новые учётные записи и удалять существующие. Доступна только Администратору (Рисунок 1.9).

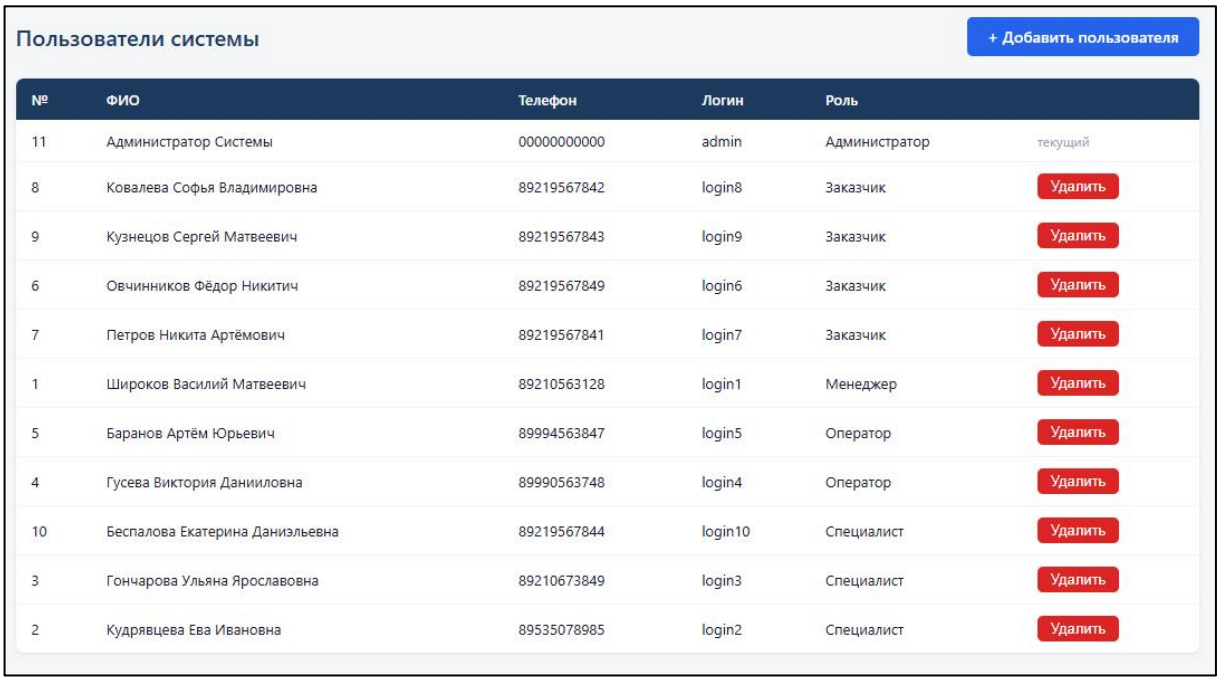

Рисунок 1.9 – Страница управления пользователями

Страница QR-кода формирует QR-код с помощью сервиса api.qrserver.com для ссылки на форму оценки качества обслуживания (Рисунок 1.10).

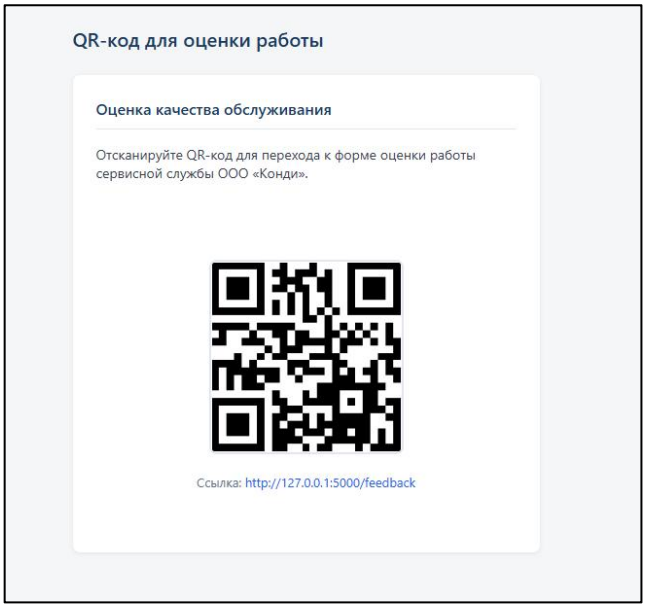

Рисунок 1.10 – Страница QR-кода

1.6 Тестирование программы

Таблица 1.1 – Результаты функционального тестирования

№	Описание теста	Ожидаемый результат	Фактический результат
1	Вход с верными данными admin/admin	Переход на dashboard	Соответствует
2	Вход с неверным паролем	Сообщение об ошибке	Соответствует
3	Доступ к /users без авторизации	Редирект на login	Соответствует
4	Создание заявки Оператором	Заявка добавлена в список	Соответствует
5	Создание заявки Заказчиком	Заявка добавлена, видна только своя	Соответствует
6	Изменение статуса заявки на «Завершена»	Дата завершения установлена автоматически	Соответствует
7	Поиск заявок по тексту	Отображены только совпадающие заявки	Соответствует
8	Фильтрация по статусу	Отображены только нужные заявки	Соответствует
9	Добавление комментария	Комментарий	Соответствует

	Специалистом	появился в карточке	
10	Удаление заявки Администратором	Заявка удалена из списка	Соответствует
11	Попытка удаления заявки Оператором	Отказ в доступе (403)	Соответствует
12	Создание нового пользователя	Пользователь появился в списке	Соответствует
13	Создание пользователя с существующим логином	Сообщение об ошибке	Соответствует
14	Удаление собственной учётной записи	Сообщение «нельзя удалить себя»	Соответствует
15	Просмотр статистики Менеджером	Данные по статусам и типам отображены	Соответствует
16	Попытка просмотра статистики Заказчиком	Пункт меню скрыт, доступ запрещён	Соответствует
17	Назначение специалиста на заявку	Специалист отображается в карточке	Соответствует
18	Резервное копирование БД (backup_db.py)	Файл резервной копии создан	Соответствует
19	Отображение QR-кода	QR-код сгенерирован и отображён	Соответствует
20	Выход из системы	Сессия завершена, редирект на login	Соответствует

По результатам тестирования все 20 тестовых случаев пройдены успешно. Процент успешно пройденных тестов составил 100%. Критических ошибок и отклонений от требований технического задания не выявлено.

1.7 Руководство системному программисту

1.7.1 Общие сведения о программе

Программа «Учёт заявок на ремонт климатического оборудования» предназначена для регистрации, хранения и обработки заявок на ремонт оргтехники и климатического оборудования. Система обеспечивает централизованный учёт заявок, распределение задач между специалистами и контроль сроков выполнения работ.

1.7.2 Функции программы

Основные функции программы:

- регистрация новых заявок (заказчиком и оператором);
- хранение информации о типе и модели оборудования, описании неисправности, заказанных комплектующих;
- назначение ответственного специалиста по заявке;
- изменение статуса заявки (новая, в процессе ремонта, выполнено, отменено и др.);
- ведение комментариев по заявке (специалисты);
- просмотр списка заявок с фильтрацией и поиском по параметрам;
- формирование простой статистики (количество выполненных заявок, среднее время ремонта, распределение по типам неисправностей);
- аутентификация пользователей по логину/паролю и разграничение прав по ролям (заказчик, оператор, специалист, менеджер по качеству).

1.7.3 Минимальный состав технических средств

Минимальная конфигурация для эксплуатации:

- персональный компьютер или сервер с архитектурой x64;
- процессор не ниже 2- ядерного, 2,0 ГГц и выше;
- оперативная память – не менее 4 ГБ (рекомендуется 8 ГБ и более);
- свободное место на диске – не менее 2 ГБ для системы, БД и логов;
- сетевое подключение (для доступа пользователей через браузер);
- монитор с разрешением не ниже 1366×768.

1.7.4 Минимальный состав программных средств

- операционная система: Windows 10/11 (или серверная версия Windows Server с поддержкой .NET 8.0);
- установленный .NET 8.0 Runtime / SDK;
- СУБД PostgreSQL версии 12 и выше;

веб-сервер (по выбору):

- IIS с модулем ASP.NET Core Hosting Bundle, либо
- встроенный Kestrel (запуск через dotnet run);
- браузер пользователя: Microsoft Edge, Google Chrome, Mozilla Firefox (актуальные версии).

1.7.5 Требования к персоналу (системному программисту)

Системный программист, сопровождающий программу, должен:

- знать основы администрирования Windows и PostgreSQL;
- разбираться в работе ASP.NET Core приложений (структура проекта, деплой, логирование);
- уметь работать с SQL (создание/изменение таблиц, резервное копирование, восстановление БД);

разбираться в механизмах аутентификации на базе cookie и настройке прав доступа;

уметь читать и анализировать журналы ошибок (лог-файлы приложения и PostgreSQL)

1.8 Структура программы

1.8.1 Сведения о структуре программы

Программа реализована как веб-приложение ASP.NET Core MVC и состоит из следующих основных слоёв:

- точка входа и конфигурация сервисов (Program.cs);
- слой доступа к данным (Entity Framework Core, AppDbContext и классы моделей);
- контроллеры (логика обработки запросов);
- модели представления (ViewModel) для передачи данных в представления;
- представления (Razor Pages, .cshtml) для отображения интерфейса пользователю;
- статические ресурсы (wwwroot при необходимости).

1.8.2 Сведения о составных частях программы

Основные составные части:

- точка входа: Program.cs Настройка пайплайна: добавление MVC, регистрации AppDbContext, настройка cookie-аутентификации и маршрутизации.
- контекст БД: Models/AppDbContext.cs Описание подключения к PostgreSQL, конфигурация сущностей Datarequest, Datauser, Datacomment, настройка связей и ключей.

Сущности (модели данных):

1) Datarequest – заявки на ремонт;

2) Datauser – пользователи системы (заказчики, операторы, специалисты, менеджеры);

3) Datacomment – комментарии по заявкам.

Контроллеры:

1) AuthController – вход/выход, определение роли по datausers.type, формирование claims и cookie.

2) HomeContoller – операции с заявками: список, поиск, создание, редактирование, удаление, статистика.

ViewModel:

1) RequestIndexViewModel – данные для главной страницы с таблицей заявок и фильтрами;

2) RequestCreateViewModel – данные для формы создания заявки;

3) RequestEditViewModel – данные для формы редактирования (статус, описание, мастер);

4) StatisticsViewModel – агрегированные данные по статистике.

Представления:

1) Views/Auth/Login.cshtml – форма входа;

2) Views/Auth/AccessDenied.cshtml – страница отказа в доступе;

3) Views/Home/Index.cshtml – список заявок, поиск и фильтрация;

4) Views/Home/Create.cshtml – создание заявки;

5) Views/Home/Edit.cshtml – редактирование заявки;

6) Views/Home/Statistics.cshtml – статистика.

1.8.2 Сведения о связях между составными частями программы

Program.cs регистрирует AppDbContext, контроллеры, аутентификацию и маршруты.

AuthController использует AppDbContext и таблицу datausers для аутентификации и назначения ролей.

HomeContoller использует AppDbContext для работы с datarequests, datausers, datacomments.

ViewModel’и наполняются данными в контроллерах и передаются в соответствующие представления.

Представления (.cshtml) используют свойства ViewModel’ей для отображения данных и генерации форм.

1.8.3 Сведения о связях с другими программами

Программа взаимодействует с сервером PostgreSQL как с внешним компонентом (через ADO.NET / Npgsql).

Возможен запуск под управлением IIS (взаимодействие с системой логирования IIS).

При необходимости может быть добавлена интеграция с другими системами (например, внешние API для уведомлений), но в базовой конфигурации таких связей нет.

1.9 Настройка программы

1.9.1 Настройка на состав технических средств

Обеспечить наличие необходимого объёма ОЗУ и свободного места на диске.

При развертывании на сервере – настроить постоянный доступ к PostgreSQL (локальный или по сети).

При работе через IIS – установить и настроить соответствующую роль/функцию, открыть нужные порты.

1.9.2 Настройка на состав программных средств

Установка драйвера.

В контексте данной программы под «драйвером» понимается:

- драйвер доступа к PostgreSQL пакет Npgsql.EntityFrameworkCore.PostgreSQL;
- при использовании внешнего COM-оборудования драйвера COM-порта, однако в текущей конфигурации приложение не использует COM-порты.

Шаги:

Установить PostgreSQL и убедиться, что сервер запущен.

Проверить наличие пакета Npgsql.EntityFrameworkCore.PostgreSQL в проекте (Project_DemoExam.csproj).

При необходимости обновить пакет через dotnet add package или NuGet.

1.10 Проверка программы

1.10.1 Описание способов проверки

Основные способы проверки корректности работы:

1) Функциональное тестирование:

- проверка сценариев входа/выхода и разграничения прав по ролям;
- проверка CRUD-операций по заявкам (создание, просмотр, изменение, удаление);
- проверка поиска и фильтрации заявок;
- проверка формирования статистики.

2) Тестирование интеграции с БД:

- проверка подключения к PostgreSQL;
- проверка корректности работы с первичными/внешними ключами и ограничениями;
- проверка автоинкремента PK и обработки ошибок 23505.

3) Тестирование отказоустойчивости:

- поведение при недоступной БД (корректное сообщение об ошибке, отсутствие падения приложения);
- корректная обработка ошибок сохранения (DbUpdateException, PostgresException).

По результатам испытаний заполняется отдельный документ «Протокол тестирования» с перечнем тест-кейсов, фактическими результатами и статусом.

1.11 Сообщения системному программисту

В процессе работы возможны следующие типичные сообщения и ситуации (таблица 1.2):

Таблица 1.2 – Сообщение, причины и решение

Сообщение / ситуация	Возможная причина	Действия системного программиста
Невозможно подключиться к базе данных	Неверная строка подключения, БД недоступна, служба PostgreSQL остановлена	Проверить строку подключения, доступность порта 5432, запустить службу PostgreSQL

Сообщение / ситуация	Возможная причина	Действия системного программиста
Исключение 23505 по *_pkey	Несогласованность sequence и фактического MAX(id), ручные вставки в БД	Проверить работу встроенного механизма ресида sequence; при необходимости выполнить setval вручную
Ошибка авторизации, сообщение «Неверный логин или пароль»	Неверно введён пароль или логин; данные в datausers некорректны	Проверить запись пользователя в datausers, при необходимости временно установить тестовый пароль
Отказ в доступе (AccessDenied / Forbid) при входе на страницу	Пользователь не имеет нужной роли или обращается к чужим данным	Проверить роли, присвоенные пользователям, логику в [Authorize] и проверках User.IsInRole
Пустой список клиентов в форме создания заявки у оператора	В datausers.type не используются ключевые слова «клиент»/«заказчик»	Уточнить значения поля type в datausers или скорректировать сопоставление типов в коде
Ошибки генерации Razor-представлений	Некорректный синтаксис Razor (@{}, скобки, строки)	Проверить указанные файлы .cshtml, исправить выражения, пересобрать проект

2 Дипломный проект

2.1 Введение дипломного проекта

Освоение системы времен – одна из самых сложных тем для школьников в курсе английского языка. Традиционные методы обучения часто не обеспечивают необходимой наглядности, интерактивности и оперативной обратной связи, что снижает интерес и эффективность усвоения материала. Внедрение цифровых инструментов, соответствующих ФГОС, позволяет создать персонализированную и вовлекающую образовательную среду. Разработка специализированного интерактивного приложения, адаптированного под школьную программу и возрастные особенности учащихся, является актуальной задачей для повышения качества обучения грамматике.

Актуальность темы заключается в необходимости создания нового программного продукта, который позволит визуализировать сложные понятия, предоставит возможность для бесконечной тренировки, обеспечит мгновенную проверку и персонализирует процесс отработки материала, тем самым повышая мотивацию и качество усвоения темы.

Цель курсового проекта – разработать интерактивное приложение для обучения временам в английском языке.

Для достижения поставленной цели необходимо решить следующие задачи:

1) провести анализ системы времен английского языка и методических требований школьной программы для формирования чёткой теоретической базы приложения;

2) исследовать и оценить существующие цифровые образовательные ресурсы и приложения, применяемые в школьной практике, с точки зрения их эффективности для изучения грамматики;

3) разработать структуру, логику и пользовательский интерфейс приложения, учитывая возрастные особенности и восприятие школьников;

4) разработать и реализовать модуль интерактивных упражнений различных типов с системой автоматической проверки и начисления баллов;

5) провести функциональное тестирование прототипа приложения на соответствие поставленным педагогическим и техническим требованиям.

2.2 Анализ предметной области

МАОУ «Средняя общеобразовательная школа №3», находящееся по адресу: Свердловская область, город Первоуральск, ул. Ватутина, 73Б. Учреждение осуществляет образовательную деятельность по основным общеобразовательным программам, включая начальное, основное и среднее общее образование. В контексте преподавания иностранных языков, в частности английского, школа следует базовой учебной программе, предусматривающей изучение английского

языка со 2 класса, с постепенным усложнением грамматического материала, включая освоение системы времен английского глагола в средних и старших классах.

Разработка интерактивного приложения для обучения временам в английском языке предназначена для решения конкретных образовательных задач в рамках школьной программы и повышения эффективности обучения английскому языку среди учащихся. В условиях современной школы, где учителя сталкиваются с большими объемами учебной нагрузки и разноуровневой подготовкой учеников, данное приложение призвано стать дополнительным инструментом, способствующим лучшему усвоению одной из самых сложных тем в курсе английского языка — системы времен.

Приложение ориентировано на учащихся средних и старших классов, которые в соответствии с федеральными государственными образовательными стандартами должны освоить основные временные формы английского глагола. Оно предназначено для поддержки основного учебного процесса, предоставляя учителям возможность организовать дополнительную практику для школьников, восполнить пробелы в знаниях и предложить индивидуальные траектории обучения с учетом разного уровня подготовки учащихся в рамках одного класса. Особую актуальность это приобретает в условиях когда в одном классе могут находиться дети с различными образовательными потребностями.

Приложение разрабатывается с учетом требований школьной программы и возрастных особенностей учащихся. Оно включает соответствующие учебному плану темы и упражнения, которые могут использоваться как на уроках под руководством учителя, так и для самостоятельной работы дома. Для школьного применения важным аспектом является простота интерфейса, доступность изложения материала и возможность интеграции с существующими образовательными процессами. В условиях ограниченного количества часов, отводимых на изучение английского языка в школе, приложение позволяет увеличить время практики и повторения материала без дополнительной нагрузки на учителя. Оно особенно ценно для отработки навыков употребления времен, которые традиционно вызывают трудности у школьников — различия между Present Perfect и Past Simple, употребление времен в условных предложениях, согласование времен в сложных предложениях. Интерактивный формат и игровые элементы способствуют повышению мотивации учащихся, что особенно важно в подростковом возрасте.

Приложение также решает задачу объективной оценки знаний — система автоматической проверки упражнений позволяет минимизировать субъективный фактор и дает возможность учителю анализировать типичные ошибки учащихся. Это помогает корректировать учебный процесс и своевременно оказывать помощь отстающим ученикам.

Таким образом, разработка данного интерактивного приложения направлена на создание эффективного, доступного и соответствующего образовательным стандартам инструмента, который поможет улучшить качество обучения английскому языку и повысить образовательные результаты школьников

2.3 Требование к своему программному продукту

Разрабатываемое приложение специально разработано для школы и ориентировано на детальную проработку времен глагола. Это позволяет учителям более целенаправленно интегрировать его в учебный процесс, а также предоставляет возможность адаптировать материал под уровень учеников, что особенно важно для эффективного обучения.

Функциональные требования:

Программный продукт должен обеспечивать выполнение следующих функций:

– возможность пользователя зарегестрироваться и авторизоваться;

– возможность выбора урока для изучения (настоящее, прошлое, будущее);

– интерактивные задания для практики (вопросы, заполнение пропусков);

– просмотр теоретического материала с примерами и объяснениями;

– система тестирования с автоматической проверкой и оцениванием;

– функция отслеживания прогресса пользователя и возврата к предыдущим занятиям.

Нефункциональные требования:

– удобный и интуитивно понятный интерфейс для пользователей всех возрастов;

– минимальное время загрузки страниц приложения;

–кроссплатформенность (работа на различных устройствах – ПК, планшеты, смартфоны);

– эффективность работы приложения при низкой скорости интернета;

Эксплуатационные требования:

–поддержка основных операционных систем (Windows, macOS, iOS, Android);

– гарантия доступности приложения не менее 95% в течение учебного года.

2.4 Обоснование выбора инструментальных средств разработки

ython - это высокоуровневый язык программирования, отличающийся эффективностью, простотой и универсальностью использования. Он широко применяется в разработке веб-приложений и прикладного программного обеспечения, а также в машинном обучении и обработке больших данных. Выбор языка Python для разработки приложения обусловлен несколькими причинами. Во-первых, Python обладает простой и читаемой синтаксической структурой, что облегчает процесс написания и понимания кода. Во-вторых, существует множество библиотек, которые позволяют легко реализовывать необходимые функции, такие как работа с графикой и веб-разработка. Кроме того, у Python есть активное сообщество, где можно получить помощь и советы.

Кроссплатформенность этого языка позволяет запускать приложение на различных операционных системах. Наконец, Python обеспечивает быструю разработку, что особенно важно для учебного проекта. По сравнению с аналогами: Java обладает более строгой типизацией и требует большего объема кода, что замедляет разработку; PHP менее универсален и больше ориентирован на веб-разработку; JavaScript, хотя и популярен для фронтенда, имеет менее последовательную архитектуру для бэкенд-разработки. Особую ценность Python представляют библиотеки для лингвистической обработки текста (NLTK, spaCy), которые могут быть использованы для анализа ответов учащихся и генерации персональных рекомендаций. Кроссплатформенность языка гарантирует работоспособность приложения в разнородной школьной ИТ-инфраструктуре.

HTML/CSS/js - это основные языки программирования, используемые для создания веб-сайтов.

Django - это свободный фреймворк для разработки быстрых и безопасных веб-приложений и сайтов на языке Python. Выбор Django обусловлен требованием создать безопасное и масштабируемое веб-приложение с разграничением прав доступа. Фреймворке предоставляет готовые решения для типовых задач образовательной платформы: система аутентификации пользователей, защита от межсайтовой подделки запросов (CSRF), валидация данных. Архитектура MTV (Model-Template-View) Django позволяет логически разделить код приложения на компоненты, что облегчает collaborative development. Встроенная админ-панель дает возможность преподавателям управлять учебным контентом без знания программирования, а механизм форм упрощает создание интерфейсов для упражнений. Выбор Django обусловлен требованием создать безопасное и масштабируемое веб-приложение с разграничением прав доступа. Он предоставляет готовые решения для типовых задач образовательной платформы: система аутентификации пользователей, защита от межсайтовой подделки запросов (CSRF), валидация данных. В сравнении с аналогами: Flask предоставляет большую гибкость, но требует ручной реализации многих компонентов; Ruby on Rails имеет схожую философию "батарейки в комплекте", но менее распространен в образовательной среде; Node.js с Express.js обеспечивает высокую производительность, но имеет менее структурированный подход к организации кода. Архитектура MTV (Model-Template-View) Django позволяет логически разделить код приложения на компоненты, что облегчает коллективную разработку.

PostgreSQL - это мощная, объектно-реляционная СУБД с открытым исходным кодом, которая обеспечивает высокую надёжность, целостность данных и соответствие стандартам SQL. Её выбор продиктован, в первую очередь, требованием к производительности при выполнении сложных аналитических запросов, необходимых для обработки данных об успеваемости учащихся, формирования детализированной статистики и построения отчётов. Механизм конкурентного контроля многоверсионности (MVCC) гарантирует согласованность данных и высокую скорость работы в условиях множественных

операций чтения и записи, что моделирует типичную нагрузку образовательной платформы.

Среди рассмотренных аналогов: Microsoft SQL Server обладает схожей мощью и развитой экосистемой, но является коммерческим продуктом, что влечёт за собой лицензионные затраты. MySQL, хотя и популярен для веб-приложений, долгое время уступал PostgreSQL в поддержке сложных SQL-операций, оконных функций и строгой проверки целостности данных, что критично для академического учёта. SQLite, идеальный для встраиваемых и однопользовательских сценариев, архитектурно не предназначен для высоконагруженных многопользовательских приложений, каким является интерактивная обучающая система.

pgAdmin - официальная графическая среда администрирования PostgreSQL - предоставляет комплексный инструментарий для визуального проектирования схемы базы данных, написания и отладки запросов, а также мониторинга производительности. Это особенно важно на этапе создания и поддержки структуры хранения для уроков, упражнений, пользователей и их прогресса.

Встроенные механизмы репликации и резервного копирования PostgreSQL обеспечивают отказоустойчивость и доступность данных, что является критическим требованием для непрерывности образовательного процесса. Для интеграции с бэкенд-фреймворком Django используется стандартный адаптер django.db.backends.postgresql, который позволяет в полной мере использовать мощь ORM Django для безопасной и эффективной работы с данными, включая миграции схемы и построение сложных запросов. Таким образом, PostgreSQL представляет собой оптимальное, масштабируемое и экономически эффективное ядро для хранения данных разрабатываемого приложения.

IDE Visual Studio Code - это редактор кода для разных языков программирования. Он относительно немного весит, гибкий и удобный. В нем можно писать, форматировать и редактировать код на разных языках. Выбор VS Code в качестве основной IDE обусловлен ее универсальностью и богатой экосистемой расширений. Этот редактор имеет интегрированную поддержку всех технологий стека: Python, Django Templates, JavaScript и SQL. Функция Live Share позволяет организовать совместную работу над проектом, а встроенный терминал с поддержкой виртуальных окружений упрощает управление зависимостями. Расширения Python и Django обеспечивают интеллектуальное завершение кода, рефакторинг и отладку, что значительно ускоряет разработку. Интеграция с Git облегчает контроль версий проекта.

2.5 Разработка архитектуры блок-схем программы

В данном разделе представлена блок-схема, отражающая общий алгоритм функционирования разработанного интерактивного веб-приложения для обучения

временам английского языка. Использование блок-схемы позволяет наглядно продемонстрировать последовательность действий пользователя, обработку данных внутри системы, а также взаимодействие основных модулей приложения. Такой подход упрощает понимание внутренней логики работы и служит основой для последующей программной реализации и улучшения функционала.

При проектировании структуры приложения была использована модульная архитектура, включающая механизмы авторизации пользователей, выбор и изучение теоретического материала по временам английского языка, выполнение интерактивных упражнений различных типов, систему проверки заданий, фиксирование прогресса и формирование статистики обучения. Блок-схема демонстрирует, как эти компоненты взаимодействуют между собой в рамках общего алгоритма: от входа пользователя в систему и выбора учебного модуля до обработки введённых ответов, сохранения результатов и отображения итоговой информации.

В блок-схеме последовательно показаны этапы получения входных данных (действий пользователя), их проверки, выполнения соответствующих операций и формирования результата, который выводится пользователю в виде оценок, подсказок или обновленного прогресса. Такая структура обеспечивает прозрачность логики работы приложения и позволяет легко расширять функционал в дальнейшем. На рисунке 2.1 приведена разработанная архитектура блок-схемы веб-приложения.

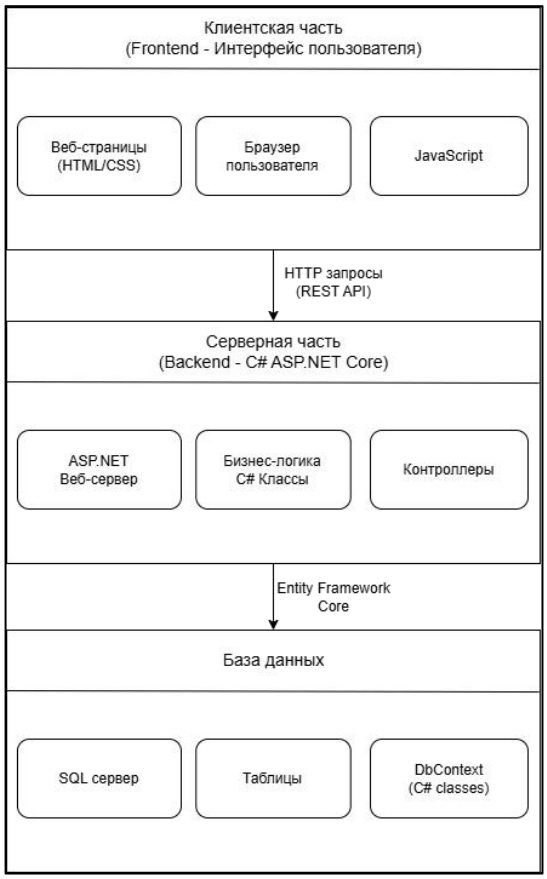

Рисунок 2.1 – Схема архитектуры веб-приложения

2.6 Разработка макетов основных форм приложения

На этапе проектирования веб-приложения важной задачей является разработка макетов пользовательских форм, которые определяют внешний вид интерфейса и структуру взаимодействия пользователя с системой. Макеты служат прототипами будущих страниц и позволяют заранее продумать логику размещения элементов управления, механизмы ввода данных и способы отображения информации. Такой подход позволяет упростить последующую реализацию интерфейса, снизить вероятность ошибок и обеспечить удобство работы операторов.

При создании макетов учитывались особенности предметной области – необходимость быстрого ввода производственных данных, точность фиксации показателей и минимизация количества действий при выполнении операций. Поэтому при разработке форм использовались принципы простоты, последовательности и интуитивной понятности. Все формы были разделены на несколько функциональных групп в соответствии с задачами системы.

Основные макеты включают:

1) форму авторизации пользователя, предназначенную для ввода логина и пароля. Макет содержит минимальный набор элементов: поля ввода, кнопку входа и ссылку для восстановления доступа. Структура формы выполнена так, чтобы обеспечить быстрый доступ к системе (Рисунок 2.2).

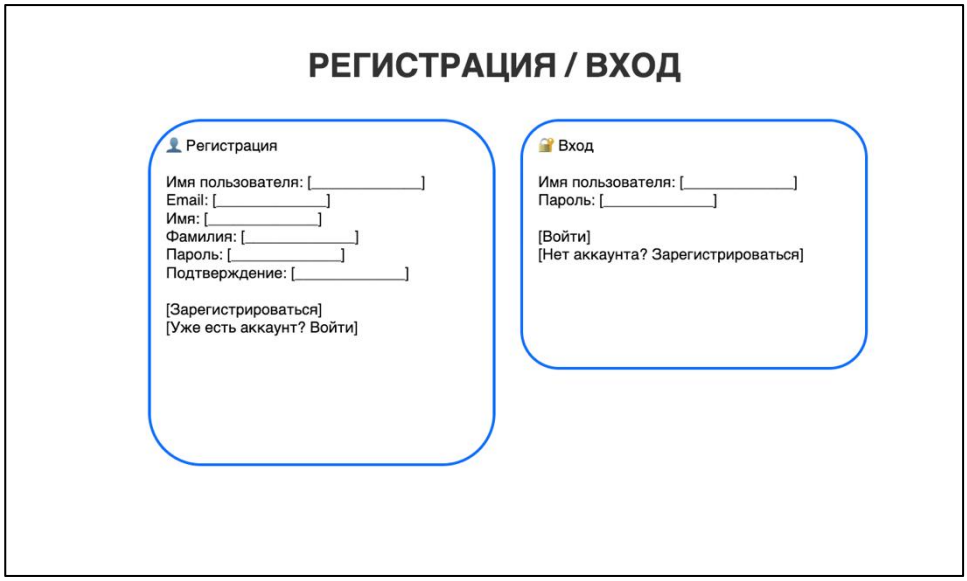

Рисунок 2.2 – Макет авторизации

2) главную страницу (панель навигации), обеспечивающую переход ко всем ключевым разделам приложения. Такой подход обеспечивает удобную навигацию и упрощает ориентацию пользователя в интерфейсе (Рисунок 2.3).

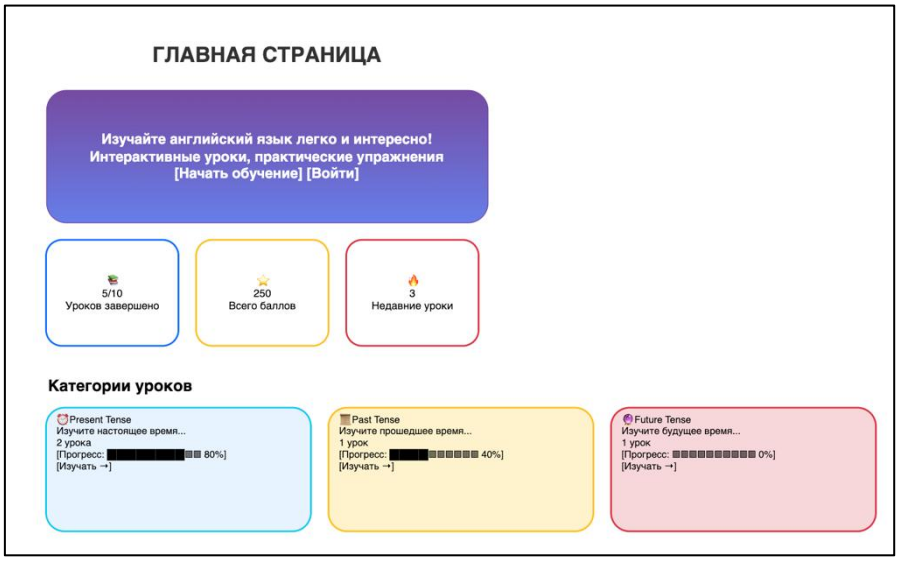

Рисунок 2.3 – Макет главной страницы

3) форма учета урока, включающая таблицу с обязательными параметрами: категория, название урока, описание, теоретический материал, примеры использования, порядок отображения и статус активности. Макет предусматривает возможность добавления, редактирования и сохранения записей в базе данных через административную панель Django. (Рисунок 2.4).

Рисунок 2.4 – Макет страницы урока

4) страница для выполнения конкретного упражнения с интерактивной формой ввода ответа и мгновенной проверкой результата. (Рисунок 2.5).

Рисунок 2.5 – Макет страницы упражнения

Каждый макет проектировался с учетом принципов адаптивности, что позволяет корректно отображать страницы как на настольных компьютерах, так и на мобильных устройствах. На основе разработанных макетов была выполнена последующая реализация интерфейса веб-приложения с использованием современных веб-технологий.

2.7 Проектирование и разработка базы данных

Проектирование базы данных является одним из ключевых этапов разработки интерактивного приложения для обучения временам английского языка. Основная цель проектирования — создать устойчивую, логичную и расширяемую структуру хранения данных, которая позволит эффективно организовать учебные материалы, упражнения, отслеживание прогресса и результаты работы пользователей.

На этапе анализа предметной области были выделены основные сущности, отражающие работу обучающего приложения. К ним относятся: категории уроков, уроки с теоретическими материалами, упражнения нескольких типов, прогресс пользователей и результаты выполнения упражнений. Эти сущности формируют основу информационной системы и обеспечивают корректное представление образовательного процесса в цифровой форме.

Структура базы данных EnglishApp включает следующие основные таблицы:

lessons_lessoncategory — категории уроков (например: Present, Past, Future):

– id: serial primary key;
– name: varchar.

lessons_lesson — учебные уроки, содержащие теорию и примеры:
– id: serial primary key;
– category_id: int foreign key;
– title: varchar;
– theory: text;
– examples: text.

lessons_exercise — упражнения различных типов, привязанные к конкретному уроку:
– id: serial primary key;
– lesson_id: int foreign key;
– type: varchar;
– question: text;
– answer_options: json / text;
– correct_answer: text.

lessons_userprogress — таблица, фиксирующая прогресс пользователя в изучении уроков:
– id: serial primary key;
– user_id: int foreign key;
– lesson_id: int foreign key;
– progress: int;
– last_access: timestamp.

lessons_userexerciseattempt — информация о попытках выполнения упражнений:
– id: serial primary key;
– user_id: int foreign key;
– exercise_id: int foreign key;
– is_correct: boolean;
– timestamp: timestamp.

Связи между таблицами реализованы через систему первичных и внешних ключей. Основные типы связей — «один-ко-многим»: одна категория включает множество уроков, каждый урок содержит множество упражнений, один пользователь имеет множество попыток выполнения упражнений и записей о прогрессе. Такая структура обеспечивает логическую целостность данных и исключает избыточность.

При проектировании схемы базы данных для EnglishApp использовались оптимальные типы данных PostgreSQL: integer, serial, varchar, text, timestamp. Модель спроектирована с учётом возможности расширения функционала в будущем — добавления новых типов упражнений, подсистемы достижений, дополнительных обучающих модулей и аналитических инструментов. Ниже на рисунке 2.5 представлена ERD-диаграмма, отражающая структуру базы данных и основные взаимосвязи между сущностями.

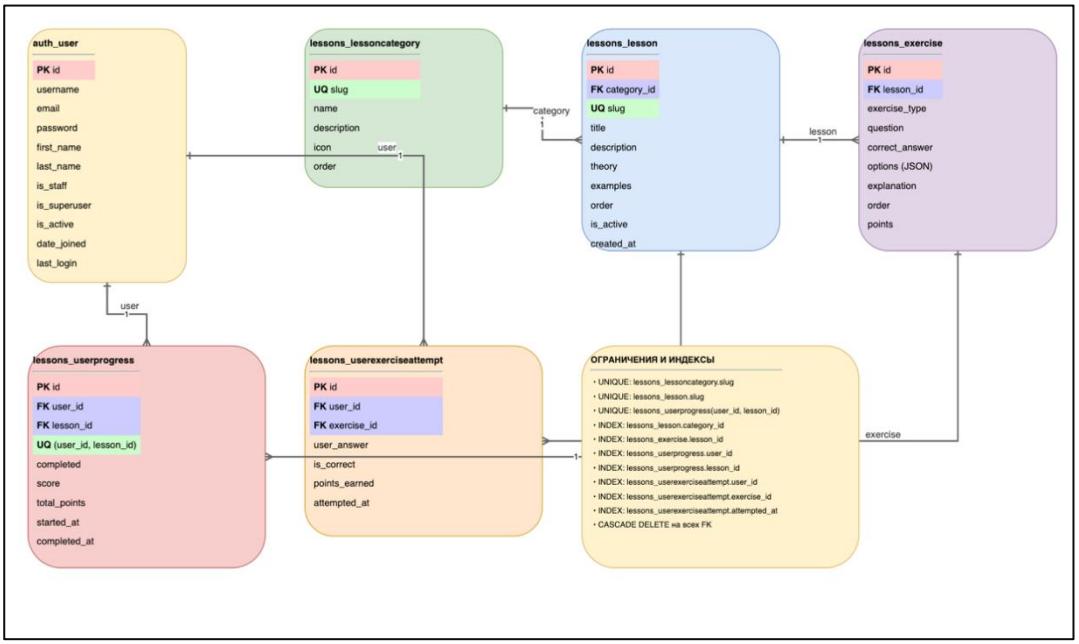

Рисунок 2.6 – ERD-диаграмма БД

2.8 Разработка пользовательского интерфейса

Пользовательский интерфейс веб-приложения EnglishApp состоит из двух основных областей: верхней навигационной панели и основной рабочей области. Навигационная панель содержит меню выбора разделов системы и информацию о пользователе. Основная область включает hero-секцию с приветствием, статистические карточки, и карточки категорий уроков с информацией о прогрессе (Рисунок 2.6).

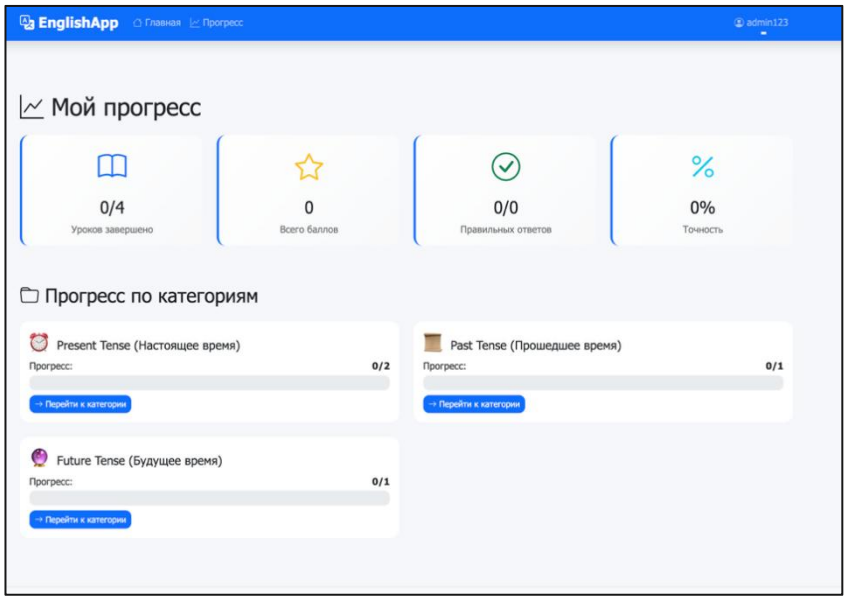

Рисунок 2.7 – Общий вид интерфейса приложения

HTML-структура основного контейнера и карточек категорий с информацией о прогрессе представлена в листинге 2.1.

Листинг 2.1 – HTML-структура основного контейнера

```
      <nav class="navbar navbar-expand-lg navbar-dark bg-
primary fixed-top">
        <div class="container">
          <a class="navbar-brand" href="/">
            <i class="bi bi-translate"></i> EnglishApp
          </a>
          <ul class="navbar-nav">
            <li class="nav-item">
              <a class="nav-link" href="/">Главная</a>
            </li>
            <li class="nav-item">
              <a class="nav-link"
href="/progress/">Прогресс</a>
            </li>
          </ul>
        </div>
      </nav>
```

Для визуального оформления элементов интерфейса использовались каскадные таблицы стилей (CSS) с применением Bootstrap 5 и кастомных стилей. Карточки категорий имеют скругленные углы, эффект тени и анимацию при наведении курсора мыши, что улучшает их восприятие. Для повышения удобства работы реализовано подсвечивание карточек при наведении курсора мыши с эффектом подъема. Основные стилевые правила представлены в листинге 2.2.

Листинг 2.2 – Ключевые CSS-стили для таблицы и модального окна

```
.card {
  border: none;
  border-radius: 15px;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 10px 25px rgba(0,0,0,.15) !important;
}

.category-card {
  border-left: 4px solid var(--info-color);
}

.stat-card {
  background: linear-gradient(135deg, #fff 0%, #f8f9fa
100%);
  border-left: 4px solid var(--primary-color);
}
/* Прогресс-бары */
.progress {
  border-radius: 10px;
  overflow: hidden;
}

.progress-bar {
  transition: width 0.6s ease;
  animation: progressBar 1s ease-out;
}

/* Кнопки */
.btn {
  border-radius: 10px;
  font-weight: 500;
  transition: all 0.3s ease;
}
.btn:hover {
  transform: translateY(-2px);

  box-shadow: 0 5px 15px rgba(0,0,0,.2);
}
```

Продолжение листинга 2.2

```
      /* Hero секция */
      .hero-section {
        background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%);
        border-radius: 0 0 30px 30px;
        margin-top: -20px;
      }
```

Основная логика работы реализована на серверной стороне с использованием Django шаблонов. При загрузке страницы данные о категориях и прогрессе пользователя загружаются из базы данных и отображаются в карточках. Код функции загрузки данных показан в листинге 2.3.

Листинг 2.3 – Функция загрузки данных с сервера

```
     total_lessons =
Lesson.objects.filter(is_active=True).count()
             completed_lessons =
UserProgress.objects.filter(
                 user=request.user,
                 completed=True
             ).count()
             total_points =
UserProgress.objects.filter(
                 user=request.user
   ).aggregate(total=Sum('score'))['total'] or 0
```

Для удобства работы с данными реализована навигация по категориям с отображением прогресса обучения. Каждая карточка категории показывает количество уроков, процент завершения и позволяет перейти к списку уроков. Функции навигации представлены в листинге 2.4.

Листинг 2.4 – Функции поиска и сортировки

```
     # Получаем прогресс для каждого урока
         for lesson in lessons:
             try:
```

Продолжение листинга 2.4

```
     progress =
UserProgress.objects.get(user=request.user,
lesson=lesson)
                 lesson.user_progress_obj = progress
                 lesson.is_completed =
progress.completed
                 lesson.user_score = progress.score
     # Вычисляем процент выполнения
      if progress.total_points > 0:

                     lesson.progress_percentage =
int((progress.score / progress.total_points) * 100)
                 else:
                     lesson.progress_percentage = 0
             except UserProgress.DoesNotExist:
                 lesson.user_progress_obj = None

                lesson.is_completed = False
                 lesson.user_score = 0
                 lesson.progress_percentage = 0}
```

Для добавления и редактирования записей используется модальное окно с формой ввода данных. Внешний вид модального окна добавления записи показан на рисунке 2.8.

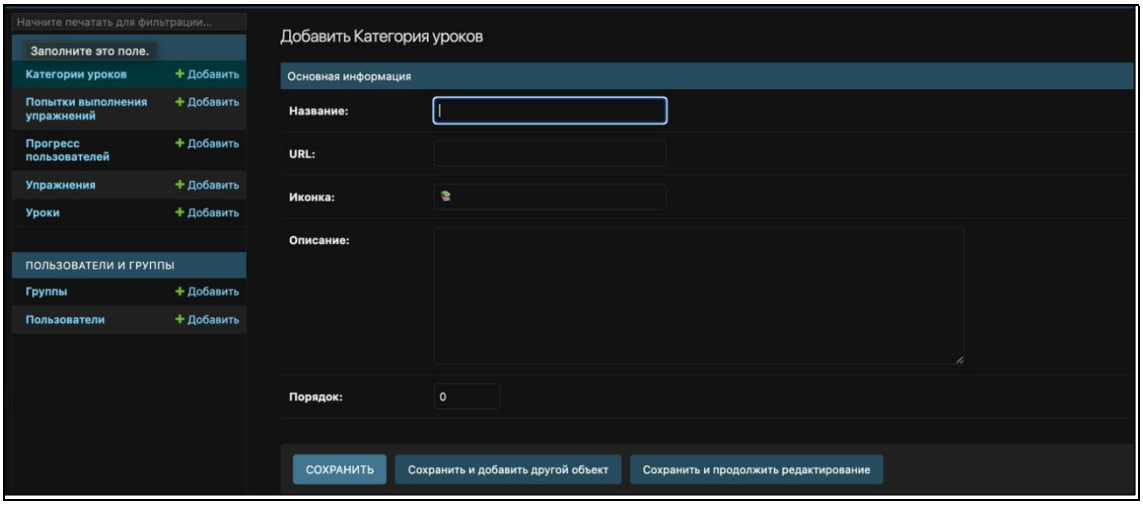

Рисунок 2.8 – Модальное окно добавления записи

Интерфейс обеспечивает удобное взаимодействие с пользователем через интуитивно понятные элементы управления, мгновенную обратную связь и защиту от некорректного ввода данных. Все операции выполняются без перезагрузки страницы, что повышает удобство работы с системой.

2.9 Доработка приложения

2.9.1 Разработка блок-схем функций

Блок-схема мини-теста с выбором слова.

Для начала необходимо создать блок-схему функции, чтобы лучше понимать принцип ее будущей работы (Рисунок 2.9).

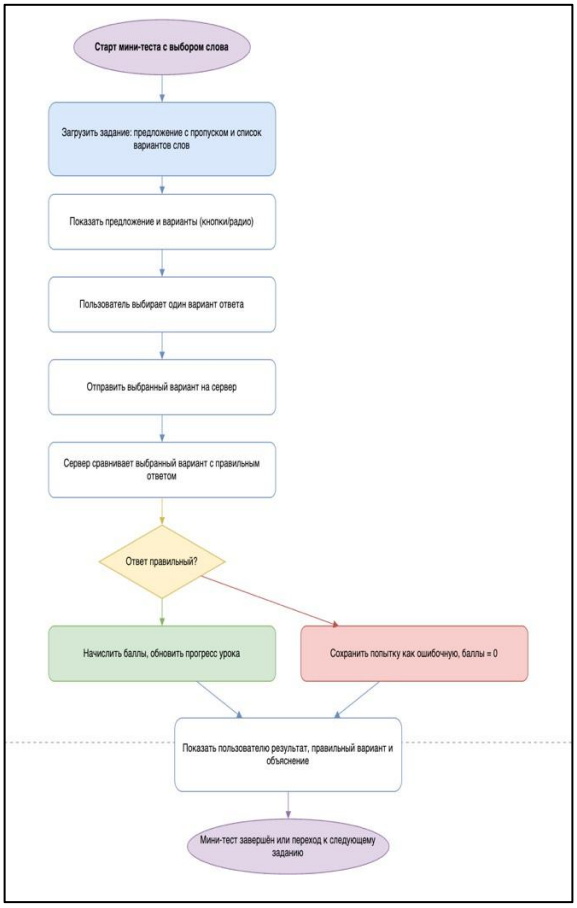

Рисунок 2.9 – Блок-схема мини-теста с выбором слова

На блок-схеме показан процесс выполнения мини-теста, где нужно выбрать правильное слово из нескольких вариантов. Сценарий начинается с загрузки задания: предложение с пропуском и списком возможных слов. Это задание отображается пользователю вместе с вариантами в виде кнопок или радиокнопок. Пользователь выбирает один вариант, после чего выбранный ответ отправляется на сервер. Сервер сравнивает этот вариант с правильным ответом и принимает решение: если ответ верен, начисляются баллы и обновляется прогресс урока; если ответ неверен, создаётся запись о неудачной попытке без баллов. Затем пользователю отображается результат: пометка

«правильно» или «неправильно», правильный вариант и объяснение. После этого мини-тест либо считается завершённым, либо пользователь переходит к следующему похожему заданию.

Блок-схема функции заполнения пропусков в предложении.

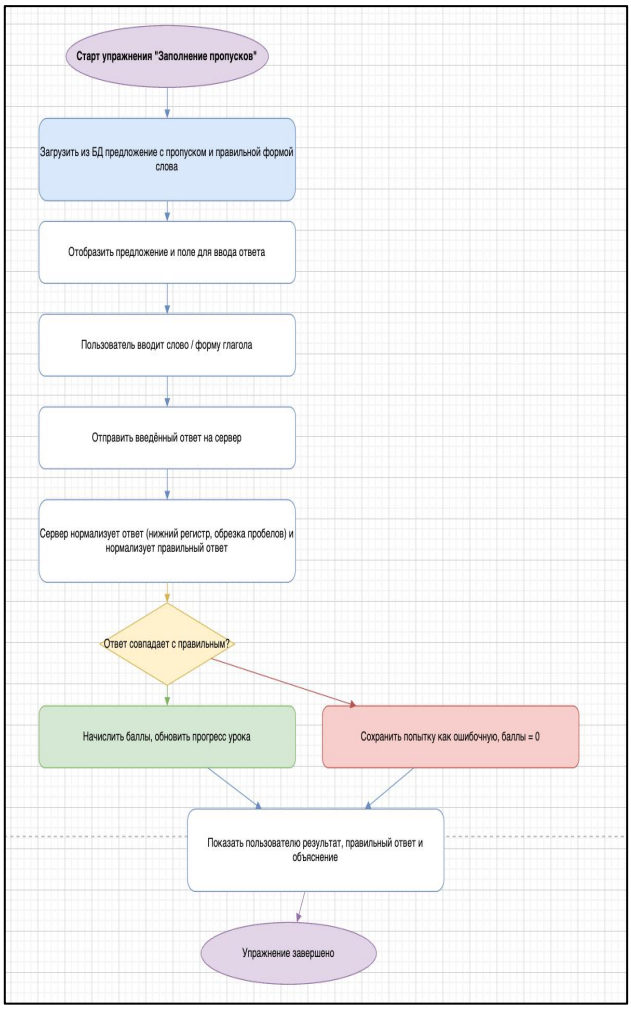

Рисунок 2.10 – Блок-схема заполнение пропусков в предложении

На блок-схеме описан процесс выполнения упражнения с заполнением пропусков в тексте. В начале система загружает из базы данных предложение с пропуском и правильной формой слова, затем показывает это предложение и поле ввода на странице. Пользователь вводит слово или форму глагола и отправляет ответ на сервер. Сервер нормализует ответ (убирает лишние пробелы, приводит к нижнему регистру) и таким же образом нормализует правильный ответ, после чего сравнивает их. Если ответы совпадают, пользователю начисляются баллы и обновляется прогресс урока; если нет — создаётся запись о неверной попытке с нулевыми баллами. В обоих случаях система возвращает на фронтенд результат, правильный ответ и объяснение, а пользователь видит итог сообщения и завершает упражнение.

Блок-схема функции игры с задачей угадать время

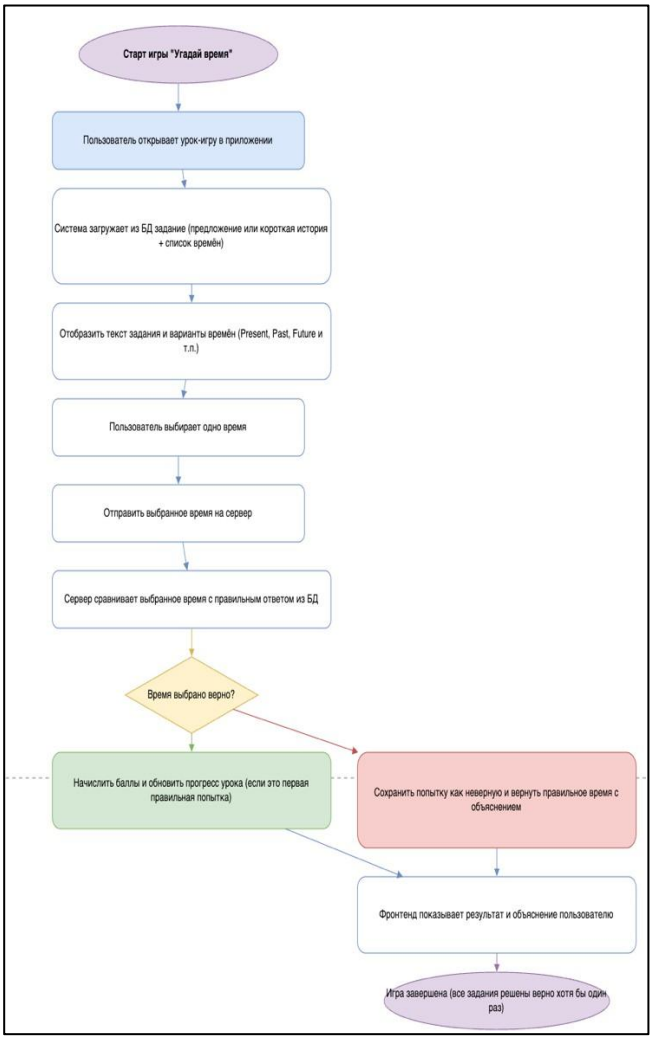

Рисунок 2.11 – Блок-схема функции угадать время

На блок-схеме изображён процесс работы мини-игры «Угадай время». Игра начинается с того, что пользователь открывает специальный урок-игру в приложении, после чего система загружает из базы данных задание: предложение или короткий текст и список возможных времён (например, Present Simple, Present Continuous, Past Simple, Future Simple и т.п.). На экране отображается текст и варианты времён, из которых пользователь выбирает один. Этот выбор отправляется на сервер, который сравнивает выбранное время с заранее сохранённым правильным ответом. Если выбор верный, начисляются баллы и при первой правильной попытке обновляется прогресс по уроку; если неверный — попытка сохраняется как ошибочная, а в ответ добавляются правильное время и объяснение. Затем фронтенд показывает пользователю результат и пояснение. Когда все задания игры хотя бы один раз решены верно, урок-игра считается полностью завершённой, и процесс игры заканчивается.

2.9.2 Разработка функционала

Разработка функции с выбором варианта ответа в предложении

В рамках модуля практических упражнений была реализована функция мини-теста с выбором слов, позволяющая пользователю тренировать грамматику и лексику в контексте предложений. Данная функция даёт возможность в интерактивном формате отрабатывать употребление правильной формы глагола или подходящего слова в предложении: система показывает фразу с пропуском, а пользователь должен выбрать один из нескольких предложенных вариантов. Мини-тесты такого типа используются как внутри обычных уроков (например, по временам), так и в игровых модулях, усиливая вовлечённость и помогая закреплять материал.

Интерфейс мини-теста с выбором слов, представленный на соответствующем экране, имеет чёткую и понятную структуру. В верхней части блока находится заголовок упражнения и краткое пояснение, под которым отображается само предложение с пропуском. Ниже располагается список вариантов ответа, каждый из которых оформлен в виде отдельного кликабельного элемента (строка списка или кнопка). Пользователь визуально видит все возможные варианты и может выбрать только один из них, что исключает неоднозначность ввода и снижает риск технических ошибок.

Каждый элемент списка помечен в явном виде: рядом с ним размещён радиобаттон или подобный индикатор выбора, что позволяет однозначно отследить, какой именно вариант был выбран. После того как пользователь принимает решение и выбирает один из вариантов, он нажимает кнопку действия (например, «Проверить ответ»). Это инициирует отправку данных на сервер: система получает идентификатор упражнения и выбранный вариант и сравнивает его с заранее заданным правильным ответом.

Если выбранный вариант совпадает с правильным, пользователь видит на экране уведомление о верном ответе, а также краткое объяснение, почему именно этот вариант считается корректным (например, привязка к правилам согласования подлежащего и сказуемого или к структуре времени). Одновременно с этим в системе обновляется прогресс пользователя: начисляются баллы, фиксируется успешная попытка и, при необходимости, пересчитывается процент выполнения урока. В случае неправильного выбора пользователю отображается сообщение об ошибке, показывается верный ответ и его объяснение, а также создаётся запись о неудачной попытке без начисления баллов. Вся логика проверки, регистрации попыток и обновления прогресса реализована в серверной части модуля упражнений и тесно связана с системой учёта достижений ученика. (Рисунок 2.12)

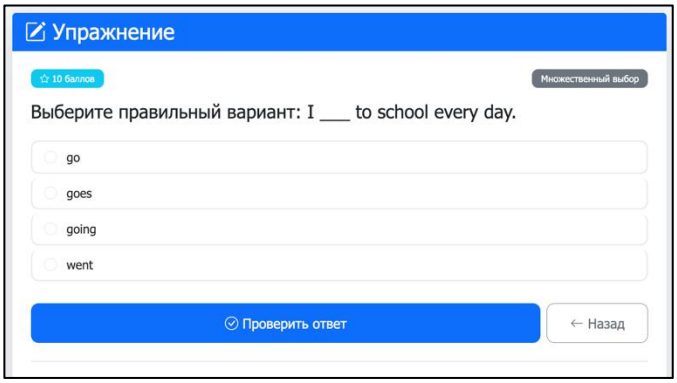

Рисунок 2.12– Поле с выбором варианта ответа

Разработка функции заполнения пропусков

Функция заполнения пропусков была добавлена в систему, чтобы пользователь мог отрабатывать точное написание слов, форм глаголов и грамматические конструкции в свободном вводе, а не только через выбор вариантов. Такое упражнение позволяет глубже проверить, насколько обучающийся действительно запомнил форму слова, а не просто узнаёт её среди готовых опций. В каждом задании пользователю предлагается предложение или короткая фраза, в которой одно из слов заменено пропуском: он должен самостоятельно вписать подходящее слово или его форму.

Интерфейс упражнения представляет собой аккуратный блок, в верхней части которого отображается текст задания с визуально выделенным пропуском (например, подчёркнутое место или пустое поле внутри текста). Непосредственно под фрагментом располагается одно крупное поле ввода, помеченное подсказкой вроде «Введите ответ» или «Введите правильную форму глагола». Это поле сконцентрировано на одном действии: пользователю не нужно выбирать среди вариантов, он полностью опирается на свои знания. При необходимости в описании задания могут быть указаны дополнительные подсказки: начальная форма глагола в скобках, время, контекст и т.д.

После того как пользователь вводит предполагаемый правильный ответ, он нажимает кнопку отправки («Проверить ответ»). В этот момент система отправляет введённый текст на сервер, где ответ проходит нормализацию: убираются лишние пробелы, строка приводится к единому регистру, а затем сравнивается с сохранённым правильным вариантом. Такой подход позволяет не наказывать за незначительные технические отличия (например, различие в верхнем и нижнем регистре), но при этом строго контролирует саму форму слова. В случае соответствия ответ признаётся правильным, пользователю начисляются баллы и обновляется общий прогресс по уроку, а на экране появляется сообщение об успехе вместе с кратким пояснением правила. Если ответ не совпадает с эталоном, система отображает уведомление о неверном ответе, показывает правильный вариант и его объяснение и при этом сохраняет в журнале факт ошибки.

Вся логика валидации и обработки этого типа упражнений реализована на стороне сервера в рамках общего модуля проверки упражнений. Она

интегрирована с системой попыток и прогресса так, чтобы каждый ввод пользователя учитывался в статистике, а накопленные баллы отражались в его личном кабинете и в показателях завершения уроков. Таким образом, упражнение «Заполнение пропусков» является важным инструментом для контроля реального владения формами и написанием английских слов. (Рисунок 2.13).

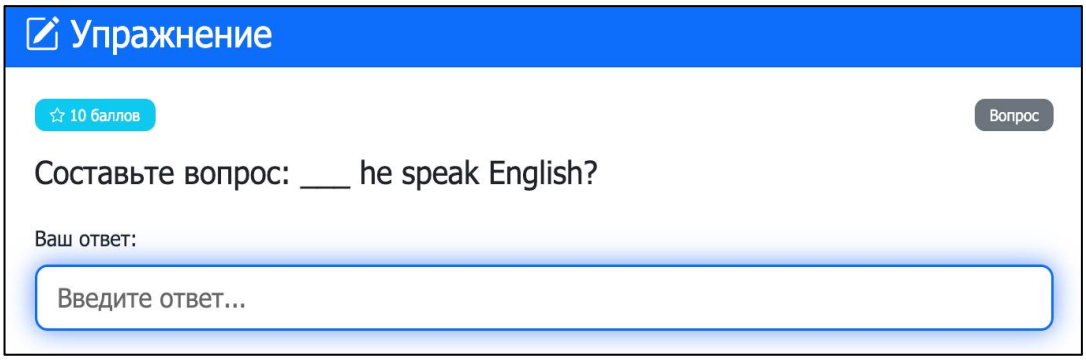

Рисунок 2.13 – Упражнение с заполнением пропусков

Разработка функции мини-игры угадывании времени

Для повышения мотивации и игровой составляющей обучения была реализована отдельная мини-игра «Угадай время». Эта функция предназначена для тренировки навыков распознавания времён в английских предложениях: пользователь видит конкретное предложение или небольшой контекст и должен определить, в каком грамматическом времени оно построено (например, Present Simple, Present Continuous, Past Simple или Future Simple). В отличие от обычных упражнений, игра оформлена как самостоятельный урок-игра с собственным интерфейсом и набором заданий, что помогает воспринимать тренировку как игровую активность, а не только как формальное тестирование.

Интерфейс игры представляет собой отдельную страницу урока, в верхней части которой располагается заголовок вроде «Игра: Угадай время» и краткое описание правил. Ниже на экране отображается основное задание: одно предложение или небольшой текст, написанный на английском языке. Под этим фрагментом выводится список возможных временных форм в виде нескольких наглядных кнопок или элементов списка: например, «Present Simple», «Present Continuous», «Past Simple», «Future Simple» и другие варианты в зависимости от настроек упражнений. Пользователь внимательно читает задание, анализирует форму глагола, наличие маркеров времени и контекст, после чего выбирает один из представленных вариантов.

При нажатии на выбранный вариант времени система отправляет данные о выборе на сервер. Серверная часть игры сравнивает выбранный пользователем вариант с правильным значением, заранее привязанным к этому предложению. Если ответ верный, пользователю начисляются баллы за конкретное задание, и, если это первая успешная попытка по данному примеру, обновляется его прогресс по уроку-игре. Одновременно формируется объяснение, которое может содержать указание на грамматические признаки: наличие

вспомогательного глагола «am/is/are» с формой на «-ing» для Present Continuous, прошедшую форму глагола для Past Simple или конструкцию «will + глагол» для Future Simple.

В случае ошибочного выбора система не только фиксирует неудачную попытку, но и возвращает пользователю подробный ответ: какое время на самом деле использовано в предложении и почему именно оно является корректным. Такой формат позволяет превратить ошибку в обучающий момент, а не просто в штраф. После показа результата игра предлагает перейти к следующему предложению из набора. Когда все задания в уроке-игре «Угадай время» будут хотя бы один раз решены правильно, система помечает этот урок как завершённый в прогрессе пользователя и может отобразить итоговое количество набранных баллов или процент правильных ответов. Вся логика последовательности заданий, проверки и обновления прогресса реализована в том же модуле упражнений, но снабжена дополнительным игровым оформлением и отдельной категорией «Игры и мини-тесты», что подчёркивает её особый статус внутри приложения. (Рисунок 2.14)

Рисунок 2.14 – Упражнение из мини-игры

Заключение

В процессе прохождения учебной практики все поставленные цели и задачи были успешно достигнуты. Основным результатом проделанной работы стала полностью готовая программа и пакет технической документации для сдачи демонстрационного экзамена, что также послужило надежной практической базой для реализации модуля дипломного проекта.

Для достижения этого результата был пройден полный цикл разработки информационной системы:

- спроектирована структура базы данных и написан функциональный программный код;
- настроена система безопасности с разграничением прав доступа пользователей по ролям;
- разработан удобный, интуитивно понятный и защищенный пользовательский интерфейс;
- составлена и оформлена вся необходимая проектная документация.

Все этапы создания проекта последовательно фиксировались в системе контроля версий Git. Итоговый программный продукт функционирует стабильно, без сбоев и эффективно решает реальные бизнес-задачи организации.

Библиография

1 Документация по платформе ASP.NET Core и языку C# [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/aspnet/core/;

2 Официальная документация СУБД PostgreSQL [Электронный ресурс] – Режим доступа: https://www.postgresql.org/docs/;

3 Руководство по Entity Framework Core для работы с базами данных [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/ef/core/;

4 Интегрированная среда разработки Visual Studio: официальное руководство [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/visualstudio/;

5 MDN Web Docs: Веб-технологии для разработчиков (HTML, CSS, JavaScript) [Электронный ресурс] – Режим доступа: https://developer.mozilla.org/ru/;

6 Руководство пользователя инструмента администрирования баз данных pgAdmin 4 [Электронный ресурс] – Режим доступа: https://www.pgadmin.org/docs/;

7 Официальная документация системы контроля версий Git [Электронный ресурс] – Режим доступа: https://git-scm.com/doc;

8 ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения [Электронный ресурс] – Режим доступа: https://docs.cntd.ru/document/1200007206.

Приложение А
(справочная)
Листинг программы

Листинг создание базы данных

```
def _import_initial_data(cursor):
users = [
('Широков Василий Матвеевич', '89210563128', 'login1', 'pass1',
'Менеджер'),
('Кудрявцева Ева Ивановна', '89535078985', 'login2', 'pass2',
'Специалист'),
('Гончарова Ульяна Ярославовна', '89210673849', 'login3',
'pass3', 'Специалист'),
('Гусева Виктория Данииловна', '89990563748', 'login4',
'pass4', 'Оператор'),
('Баранов Артём Юрьевич', '89994563847', 'login5', 'pass5',
'Оператор'),
('Овчинников Фёдор Никитич', '89219567849', 'login6', 'pass6',
'Заказчик'),
('Петров Никита Артёмович', '89219567841', 'login7', 'pass7',
'Заказчик'),
('Ковалева Софья Владимировна', '89219567842', 'login8',
'pass8', 'Заказчик'),
('Кузнецов Сергей Матвеевич', '89219567843', 'login9', 'pass9',
'Заказчик'),
('Беспалова Екатерина Даниэльевна', '89219567844', 'login10',
'pass10', 'Специалист'),
('Администратор Системы', '00000000000', 'admin', 'admin',
'Администратор'),
]
for fio, phone, login, password, role in users:
cursor.execute(
'INSERT INTO users (fio, phone, login, password, role) VALUES
(?,?,?,?,?)',
(fio, phone, login, hash_password(password), role)
)
requests_data = [
(1, '2023-06-06', 'Кондиционер', 'TCL TAC-12CHSA/TPG-W белый',
'Не охлаждает воздух', 'В процессе ремонта', None, None, 2, 7),
(2, '2023-05-05', 'Кондиционер', 'Electrolux EACS/I-
09HAT/N3_21Y белый',
'Выключается сам по себе', 'В процессе ремонта', None, None, 3,
8),
(3, '2022-07-07', 'Увлажнитель воздуха', 'Xiaomi Smart
Humidifier 2',
'Пар имеет неприятный запах', 'Готова к выдаче', '2023-01-01',
None, 3, 9),
(4, '2023-08-02', 'Увлажнитель воздуха', 'Polaris PUH 2300
WIFI IQ Home',
```

```
        'Увлажнитель воздуха продолжает работать при предельном
снижении уровня воды',
        'Новая заявка', None, None, None, 8),
        (5, '2023-08-02', 'Сушилка для рук', 'Ballu BAHD-1250',
        'Не работает', 'Новая заявка', None, None, None, 9),
        ]
        for r in requests_data:
        cursor.execute(
        '''INSERT INTO requests
        (request_id, start_date, climate_tech_type, climate_tech_model,
        problem_description,      request_status,      completion_date,
repair_parts,
        master_id, client_id)
        VALUES (?,?,?,?,?,?,?,?,?,?)''', r
        )
        comments_data = [
        (1, 'Всё сделаем!', 2, 1),
        (2, 'Всё сделаем!', 3, 2),
        (3, 'Починим в момент.', 3, 3),
        ]
```